

Week-9-s

Session-17

Task-1

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 50

typedef enum {
    RUNNING,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    char command[100];
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

/* Convert state to text */
const char *get_state(JobState state) {
    if (state == RUNNING)
        return "RUNNING";
    else
        return "COMPLETED";
}

/* Update state of all jobs */
void update_states() {
    int status;

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state == RUNNING) {
            pid_t result = waitpid(
                jobs[i].pid,
                &status, 0);

            if (!active) {
                printf("No active jobs.\n");
            }

            printf("=====\n");
        }
    }

    /* Display all jobs */
    void list_all_jobs() {
        update_states();

        printf("\n===== ALL JOBS =====\n");

        printf("%-8s %-10s %-20s %-12s\n",
            "Job ID",
            "PID",
            "Command",
            "Status");

        printf("-----\n");

        for (int i = 0; i < job_count; i++) {
            printf("%-8d %-10d %-20s %-12s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command,
                get_state(jobs[i].state));
        }

        printf("=====\n");
    }

    int main() {
        char command[100];

        while (1) {
```

```

    }

    if (strlen(command) == 0)
        continue;

    /* Create child process */
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        continue;
    }

    if (pid == 0) {
        execl("/bin/sh", "sh", "-c", command, NULL);

        perror("exec");
        exit(1);
    }

    /* Store job information */
    if (job_count < MAX_JOBS) {

        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;
        strcpy(jobs[job_count].command, command);
        jobs[job_count].state = RUNNING;

        printf("Job [%d] started with PID %d\n",
               jobs[job_count].job_id,
               pid);

        job_count++;
    }
}

| return 0;
}

```

```

root@Shamikahasini:~# nano joblist.c
root@Shamikahasini:~# gcc joblist.c -o joblist
root@Shamikahasini:~# ./joblist

```

```

joblist> sleep 20
Job [1] started with PID 810

```

```

joblist> sleep 30
Job [2] started with PID 816

```

```

joblist> sleep 40
Job [3] started with PID 820

```

```

joblist> jobs

```

```

===== ACTIVE JOBS =====
Job ID   PID      Command      Status
-----
2         816      sleep 30     RUNNING
3         820      sleep 40     RUNNING
=====

```

```

joblist> all

```

```

===== ALL JOBS =====
Job ID   PID      Command      Status
-----
1         810      sleep 20     COMPLETED
2         816      sleep 30     COMPLETED
3         820      sleep 40     RUNNING
=====

```

```

joblist> jobs

```

```

===== ACTIVE JOBS =====
Job ID   PID      Command      Status
-----
3         820      sleep 40     RUNNING
=====

```

```

joblist> |

```

Observation:

The program successfully lists active jobs with their Job IDs, PIDs, commands and status. The job state is updated automatically when a process completes. The formatted job table makes the process information easy to retrieve and validate.

Task-2

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>

int main() {
    char command[100];
    pid_t pid;
    int status;

    printf("===== FOREGROUND JOB CONTROL =====\n");

    printf("Enter command to run in foreground: ");
    fflush(stdout);

    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Create a new process group */
        setpgid(0, 0);

        printf("\nChild Process Started\n");
        printf("PID : %d\n", getpid());
        printf("PGID : %d\n", getpgrp());

        /* Execute command */
        execl("/bin/sh", "sh", "-c", command, NULL);

        perror("exec");
    }

    /* Transfer terminal control to the target job. */
    if (tcsetpgrp(STDIN_FILENO, pid) == -1) {
        perror("tcsetpgrp");
    } else {
        printf("Terminal control transferred to job.\n");
    }

    printf("Waiting for foreground job to complete...\n");

    /* Wait for the foreground process. */
    waitpid(pid, &status, 0);

    /* Take terminal control back. */
    if (tcsetpgrp(STDIN_FILENO, getpgrp()) == -1) {
        perror("tcsetpgrp");
    } else {
        printf("Terminal control returned to shell.\n");
    }

    if (WIFEXITED(status)) {
        printf("Job completed normally.\n");
        printf("Exit Status: %d\n", WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status)) {
        printf("Job terminated by signal %d\n",
            WTERMSIG(status));
    }

    printf("Job state updated: COMPLETED\n");

    return 0;
}
```

GNU nano 7.2 foreground.c *

```

root@Shamikahasini:~# nano foreground.c
root@Shamikahasini:~# gcc foreground.c -o foreground
root@Shamikahasini:~# ./foreground
===== FOREGROUND JOB CONTROL =====
Enter command to run in foreground: sleep 5

Parent Process
Target Job PID : 964
Target Job PGID : 964
Terminal control transferred to job.
Waiting for foreground job to complete...

Child Process Started
PID : 964
PGID : 964

[1]+  Stopped                  ./foreground
root@Shamikahasini:~# ./foreground
===== FOREGROUND JOB CONTROL =====
Enter command to run in foreground: echo "Foreground Job Running"

Parent Process
Target Job PID : 967
Target Job PGID : 967
Terminal control transferred to job.
Waiting for foreground job to complete...

Child Process Started
PID : 967
PGID : 967
Foreground Job Running

[2]+  Stopped                  ./foreground
root@Shamikahasini:~# ./foreground
===== FOREGROUND JOB CONTROL =====
Enter command to run in foreground: |

```

Observation:

The target foreground job was successfully identified and assigned a process group. Terminal control was transferred to the foreground process, and the parent waited for its completion. After completion, terminal control was returned to the parent and the job state was updated successfully.

Session - 18

Task-1

To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior.

```
GNU nano 7.2
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Signal received: %d\n", sig);
}

int main() {
    signal(SIGINT, handler);

    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C\n");

    while (1)
        sleep(1);

    return 0;
}
```

```
root@Shamikahasini:~# nano signal.c
root@Shamikahasini:~# gcc signal.c -o signal
root@Shamikahasini:~# ./signal
PID: 1603
Press Ctrl+C
^CSignal received: 2
```

```
root@Shamikahasini:~# ps
  PID TTY          TIME CMD
 1608 pts/2    00:00:00 bash
 1626 pts/2    00:00:00 ps
root@Shamikahasini:~# kill PID
-bash: kill: PID: arguments must be process or job IDs
root@Shamikahasini:~# kill -SIGINT PID
-bash: kill: PID: arguments must be process or job IDs
root@Shamikahasini:~# ps -p PID
error: process ID list syntax error

Usage:
ps [options]

Try 'ps --help <simple|list|output|threads|misc|all>'
or 'ps --help <s|l|o|t|m|a>'
for additional help text.

For more details see ps(1).
root@Shamikahasini:~#
```

Observation

- The process ID (PID) of a running process was identified using the ps command.
- Signals such as **SIGINT** and **SIGTERM** were sent to processes using the kill command.
- The effect of signals on process execution was observed.
- The experiment demonstrated how the **terminal and operating system communicate with processes using signals**.
- Process status was verified using ps -p <PID>.

Task-2

To resume stopped jobs, send continue signals, update job status, maintain the job table, verify execution, and test background recovery.

```
root@Shamikahasini:~# sleep 100 &
[1] 1645
root@Shamikahasini:~# jobs
[1]+  Running                  sleep 100 &
root@Shamikahasini:~# kill -STOP %1
root@Shamikahasini:~# jobs
[1]+  Stopped                  sleep 100
root@Shamikahasini:~# kill -CONT %1
root@Shamikahasini:~# jobs
[1]+  Running                  sleep 100 &
root@Shamikahasini:~# ps
  PID TTY          TIME CMD
 1608 pts/2    00:00:00 bash
 1645 pts/2    00:00:00 sleep
 1649 pts/2    00:00:00 ps
root@Shamikahasini:~# |
```

Observation:

A background job was successfully started, stopped using **SIGSTOP**, and resumed using **SIGCONT**. The jobs and ps commands were used to verify the job status and execution. This demonstrated how stopped background processes can be recovered and continued successfully.